

Instructivo — Configuración y aclaraciones

Servicio de email

El práctico requiere un servicio de email real para el envío de los links de verificación y recuperación de contraseña. Las credenciales se configuran en el archivo `back/.env`.

Variables de entorno

Agregar al `.env` del backend:

```
# Resend (recomendado por simplicidad)
RESEND_API_KEY=re_XXXXXXXXXXXX

# O SMTP
SMTP_HOST=smtp.mailtrap.io
SMTP_PORT=587
SMTP_USER=tu_usuario
SMTP_PASS=tu_contraseña
SMTP_FROM=noreply@tudominio.com
```

El `.env` no se sube al ZIP de entrega.

Servicios recomendados

Servicio	Gratuito	Notas
Resend	100 emails/día	Solo requiere API key
SMTP propio	—	Cualquier servidor SMTP

Preguntas frecuentes

¿Qué pasa si el token de verificación expira?

El token de verificación de email **no expira** (es válido hasta que se usa o se reemite uno nuevo). El token de recuperación de contraseña sí expira (1 hora).

¿Dónde van las rutas de los endpoints de perfil?

Los endpoints `PATCH /users/me/password` y `PATCH /users/me/email` ya existen en el backend. No es necesario crearlos, solo consumirlos desde el frontend.

backend. No es necesario crearlos, solo consumirlos desde el frontend.

¿El frontend se entrega funcionando?

El frontend se entrega con login, register, profile, guards e interceptores funcionando. Las páginas nuevas (`/verify-pending` , `/verify-email` , `/forgot-password` , `/reset-password`) deben crearse. Los componentes existentes (`register` , `profile` , `login`) deben modificarse según las consignas.

¿Se puede usar Bootstrap para los toasts?

Sí. Bootstrap 5 ya está incluido en el proyecto. Pueden usar las clases `.toast` de Bootstrap o armar un componente propio.

Comandos útiles

```
# Ver logs del backend
cd back && npm run start:dev

# Ver logs del frontend
cd front && npm start

# Servidor MCP (solo promocionar)
cd mcp && npx tsx src/index.ts
```

Servidor MCP (promocionar)

Estructura del template

```
mcp/
├── package.json          # Dependencias y scripts
├── tsconfig.json         # Configuración de TypeScript
├── src/
│   ├── index.ts          # Punto de entrada del servidor
│   ├── api-client.ts      # Cliente HTTP para consumir back
│   ├── tool-factory.ts    # Utilidad para registrar tools
│   └── tools/
│       ├── index.ts       # Índice que agrupa todas las tools
│       └── auth.ts        # Tools de autenticación (ejemplo)
```

Dependencias

El `package.json` ya incluye `@modelcontextprotocol/sdk` , `axios` y `tsx` . **Importante:**

instalar también `zod` (falta en el template):

```
cd mcp && npm install && npm install zod
```

Cómo agregar una tool nueva

Cada tool se define como un objeto `ToolDef` :

```
{
  name: "nombre_de_la_tool",
  description: "Descripción clara de lo que hace",
  inputSchema: {
    param1: z.string(),
    param2: z.number().int().optional(),
  },
  handler: async (args) => api.get("/ruta", args),
}
```

Pasos:

1. Crear el archivo en `src/tools/` (ej. `products.ts` , `categories.ts`)
2. Exportar un array de `ToolDef[]` (ver `auth.ts` como ejemplo)
3. Importarlo y agregarlo al array en `src/tools/index.ts`

Tools ya implementadas (auth)

Tool	Descripción
<code>auth_login</code>	Inicia sesión y guarda el token JWT
<code>auth_register</code>	Registra un nuevo usuario y guarda el token
<code>auth_me</code>	Devuelve el usuario autenticado
<code>delete_my_account</code>	Elimina la cuenta del usuario autenticado

Tools a implementar

Ver la sección **2. Condiciones para Promocionar** del práctico. Deben crearse:

- `src/tools/products.ts` — 5 tools (CRUD de productos)
- `src/tools/categories.ts` — 5 tools (CRUD de categorías)
- `src/tools/users.ts` — 4 tools (usuarios, roles, password, email)

Cómo probar

```
cd mcp && npx tsx src/index.ts
```

El servidor se conecta por STDIO. Para probar las tools desde opencode, configurar `opencode.json` como se indica en el práctico. También se puede probar con herramientas como `mcp-cli` o conectándolo a Claude Desktop.

Autenticación automática

Si se definen las variables de entorno `API_C_EMAIL` y `API_C_PASSWORD`, el servidor hace login automático al iniciar. De lo contrario, debe invocarse `auth_login` manualmente antes de usar el resto de las tools